

Web Application Development

Lecture 12: Middleware, CORS & Error Handling

SWE 381 – Web Application Development

Dr. Ohoud Alharbi Dr. Achraf Gazdar

Department of Software Engineering
College of Computer and Information Sciences
King Saud University

Term 2 – 2025/2026

Learning Objectives

By the end of this lecture, you will be able to:

- Explain the **middleware pipeline** and how requests flow through it
- Write functions with the correct **(req, res, next)** signature
- Build a custom **logger middleware** that logs every request
- Configure **CORS** to allow your React frontend to call the API
- Create **input validation middleware** that guards route handlers
- Implement a **global error handler** with the 4-parameter signature
- Use the **next(err)** pattern to pass errors down the pipeline
- Order middleware **correctly** in index.js to avoid common bugs

Agenda

- 1 Middleware Pipeline Concept
- 2 Custom Logger Middleware
- 3 CORS Configuration
- 4 Input Validation Middleware
- 5 Global Error Handler & `next(err)`
- 6 Correct Middleware Order
- 7 Recap

Outline

- 1 Middleware Pipeline Concept
- 2 Custom Logger Middleware
- 3 CORS Configuration
- 4 Input Validation Middleware
- 5 Global Error Handler & `next(err)`
- 6 Correct Middleware Order
- 7 Recap

What is Middleware?

W3Schools – Node.js Middleware

Middleware is a key part of Node.js web applications, particularly in Express.js. It provides a way to **add and reuse common functionality** across your application's routes and endpoints. Think of middleware as a series of processing layers that requests pass through – like an **assembly line for HTTP requests**.

What middleware functions can do:

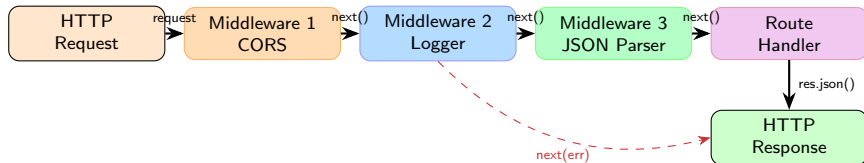
- Execute any code
- Modify req and res objects
- End the request-response cycle (send a response)
- Call `next()` to pass control to the next middleware

If `next()` is not called

If the current middleware does not send a response and does not call `next()`, the request will be **left hanging** – the client times out.

```
1 // Minimal middleware anatomy
2 app.use((req, res, next) => {
3   // 1. Do something
4   console.log('Middleware ran!');
5
6   // 2. Decide what to do next:
7   //   a) Call next() to continue
8   next();
9   //   b) Or send a response to stop
10  //   res.json({ error: 'Stop here' });
11 });
```

The Middleware Pipeline – Visual



W3Schools – Middleware Pipeline

Middleware functions are executed **in the order they are defined**, creating a pipeline through which requests flow. Each middleware function can perform operations on the request and response objects and decide whether to **pass control to the next middleware** or end the request-response cycle.

(req, res, next) – The Three Arguments

```
// The three arguments explained
app.use((req, res, next) => {

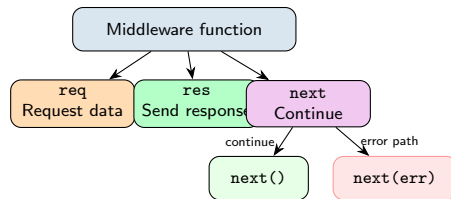
  // req = incoming HTTP request
  // Contains: method, url, params,
  //           query, body, headers
  console.log(req.method); // "GET"
  console.log(req.url);    // "/api/courses"

  // res = outgoing HTTP response
  // Methods: .json(), .send(),
  //           .status(), .redirect()
  // (use to SEND a response)

  // next = function to continue pipeline
  // Call next()      => continue to next middleware
  // Call next(error) => jump to error handler

  next(); // pass control forward
});
```

```
// Route handlers are ALSO middleware!
// They are just the LAST step
app.get('/api/courses', (req, res) => {
  // This is middleware that ends the cycle
  res.json(courses); // no next() needed
});
```



💡 next vs next(err)

`next()` moves to the next normal middleware.
`next(err)` skips all normal middleware and jumps straight to the error handler.

W3Schools Middleware Example – Pipeline Demo

```
const express = require('express');
const app = express();

// Middleware 1 -- runs for EVERY request
app.use((req, res, next) => {
  console.log('Middleware 1: This always runs');
  next();
});

// Middleware 2 -- runs for EVERY request
app.use((req, res, next) => {
  console.log('Middleware 2: This also runs');
  next();
});

// Route handler -- last in the chain
app.get('/', (req, res) => {
  res.send('Hello World!');
});

app.listen(8080, () => {
  console.log('Server running on port 8080');
});
```

Console when GET / is requested

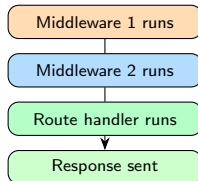
Middleware 1: This always runs

Middleware 2: This also runs

Browser response

Hello World!

Execution order:



Outline

- 1 Middleware Pipeline Concept
- 2 Custom Logger Middleware
- 3 CORS Configuration
- 4 Input Validation Middleware
- 5 Global Error Handler & `next(err)`
- 6 Correct Middleware Order
- 7 Recap

Custom Logger Middleware

W3Schools – Middleware Concept

Here is an example of a middleware function called **requestTime**: it attaches the current time to the request object. `app.use(requestTime)` registers it for all requests.

```
1 // middleware/logger.js
2 const logger = (req, res, next) => {
3   const method    = req.method;
4   const url       = req.url;
5   const timestamp = new Date().toISOString();
6
7   // Attach to req so later handlers can use it
8   req.requestTime = Date.now();
9
10  console.log(
11    `[${timestamp}] ${method} ${url}`
12  );
13
14  // MUST call next() to continue the pipeline
15  next();
16 };
17
18 module.exports = logger;
```

Using the logger in index.js:

```
1 const logger = require('./middleware/logger');
```

Console output for each request

```
[2025-02-01T10:20:31Z] GET /api/courses
[2025-02-01T10:20:35Z] POST /api/courses
[2025-02-01T10:21:00Z] DELETE /api/courses/3
```

💡 Morgan for production

In production, use the **Morgan** npm package instead of a custom logger:

```
npm install morgan
```

```
app.use(require('morgan')('dev'))
```

Morgan provides formatted logs with response time and status codes.

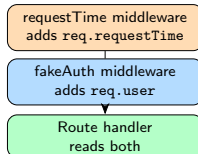
Attaching Data to req – Middleware Communication

Middleware can add properties to req:

```
1 // Middleware: attach timestamp to req
2 const requestTime = (req, res, next) => {
3   req.requestTime = Date.now();
4   next();
5 };
6
7 // Middleware: attach a fake user to req
8 // (in real apps this comes from a JWT)
9 const fakeAuth = (req, res, next) => {
10   req.user = {
11     id: 1,
12     name: 'Ahmed',
13     role: 'student'
14   };
15   next();
16 };
17
18 app.use(requestTime);
19 app.use(fakeAuth);
20
21 // Route handler -- uses what middleware added
22 app.get('/api/me', (req, res) => {
23   res.json({
24     user: req.user,           // from fakeAuth
25     requestedAt: req.requestTime // from requestTime
26   });
27 });
```

GET /api/me – 200

```
{
  "user": {
    "id": 1,
    "name": "Ahmed",
    "role": "student"
  },
  "requestedAt": 1706783231000
}
```



Outline

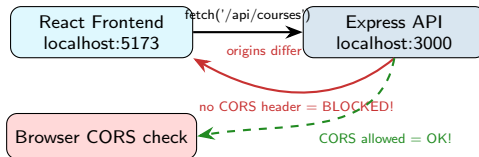
- 1 Middleware Pipeline Concept
- 2 Custom Logger Middleware
- 3 CORS Configuration**
- 4 Input Validation Middleware
- 5 Global Error Handler & `next(err)`
- 6 Correct Middleware Order
- 7 Recap

What is CORS and Why Does it Block Requests?

CORS – Cross-Origin Resource Sharing

CORS errors occur when a web browser **blocks requests from one origin to another**. This is a **browser security feature** that prevents malicious websites from making requests to your API on behalf of unsuspecting users. The error message looks like:

Access to fetch blocked by CORS policy: No 'Access-Control-Allow-Origin' header



💡 CORS is a BROWSER feature

Postman, curl, and other server-to-server calls are **never blocked by CORS**. Only browser-side JavaScript is affected. That's why Postman works but your React app gets blocked.

CORS Configuration with the cors Package

```
$ npm install cors
```

1. Allow ALL origins (development only):

```
1 const cors = require('cors');
2
3 // WARNING: Only for local development!
4 app.use(cors());
```

2. Allow specific origins (recommended):

```
1 const corsOptions = {
2   origin: 'http://localhost:5173',
3   methods: ['GET', 'POST', 'PUT', 'DELETE'],
4   allowedHeaders: [
5     'Content-Type',
6     'Authorization'
7   ]
8 };
9
10 app.use(cors(corsOptions));
```

3. Whitelist multiple origins:

```
1 const corsOptions = {
2   origin: [
3     'http://localhost:5173', // React dev
4     'https://swe381-app.com' // production
5   ]
6 };
```

Option	Purpose
origin	Allowed frontend URLs
methods	Allowed HTTP verbs
allowedHeaders	Allowed req headers
credentials	Allow cookies/JWT

⚠ Never use origin: '*' in production

origin: '*' allows **any website** to call your API. Use specific origins in production, especially if your API handles user data.

💡 Environment-based CORS

Read the allowed origin from .env:

FRONTEND_URL=http://localhost:5173

CORS with Environment Variables

.env file:

```
PORT=3000
NODE_ENV=development
FRONTEND_URL=http://localhost:5173
```

middleware/corsConfig.js:

```
1 const cors = require('cors');
2
3 const corsOptions = {
4   origin: process.env.FRONTEND_URL,
5   methods: ['GET', 'POST', 'PUT', 'DELETE'],
6   allowedHeaders: [
7     'Content-Type',
8     'Authorization'
9   ],
10  credentials: true
11 };
12
13 module.exports = cors(corsOptions);
```

index.js:

```
1 require('dotenv').config();
2 const corsMiddleware =
3   require('./middleware/corsConfig');
4
5 // CORS must be the FIRST middleware
```

Before CORS:

React browser fetch fails

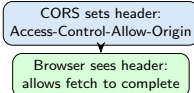
```
ERROR: Access to fetch at
'http://localhost:3000/api/courses'
from origin
'http://localhost:5173' blocked
by CORS policy
```

After CORS configured:

React browser fetch works

```
HTTP/1.1 200 OK
Access-Control-Allow-Origin:
http://localhost:5173

[array of courses returned]
```



Outline

- 1 Middleware Pipeline Concept
- 2 Custom Logger Middleware
- 3 CORS Configuration
- 4 Input Validation Middleware**
- 5 Global Error Handler & `next(err)`
- 6 Correct Middleware Order
- 7 Recap

Input Validation Middleware

Input Validation Pattern

Input validation middleware sits **between the route definition and the route handler**. It checks `req.body` for required fields and calls `next()` if valid, or sends a 400 error if invalid – without touching the controller logic.

```
1 // middleware/validateCourse.js
2 const validateCourse = (req, res, next) => {
3   const { title, dept, credits } = req.body;
4
5   const errors = [];
6
7   if (!title || !title.trim()) {
8     errors.push('title is required');
9   }
10  if (!dept || !dept.trim()) {
11    errors.push('dept is required');
12  }
13  if (credits !== undefined &&
14      (isNaN(credits) || credits < 1)) {
15    errors.push('credits must be >= 1');
16  }
17
18  if (errors.length > 0) {
19    // Block the request -- don't call next()
20    return res.status(400).json({
21      error: 'Validation failed',
22      messages: errors
23    });
24  }
25
26  // All valid -- continue to route handler
27  next();
28 }
```

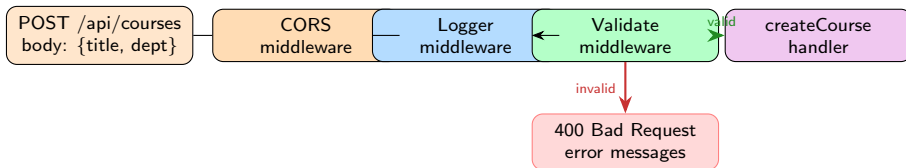
Applying the middleware to a route:

```
1 const validateCourse =
2   require('../middleware/validateCourse');
3
4 // validateCourse runs BEFORE the handler
5 router.post(
6   '/',
7   validateCourse,      // guard
8   createCourse         // handler (only if valid)
9 );
10
11 router.put(
12  ('/:id'),
13   validateCourse,      // same guard
14   updateCourse
15 );
```

POST missing title – 400

```
{
  "error": "Validation failed",
  "messages": ["title is required"]
}
```

Validation Middleware – Route Flow



Reusable guards

Validation middleware is **reusable**: the same `validateCourse` can guard both the POST and PUT routes. Write it once, use it in many places.



Separation of concerns

Keeping validation **out of the controller** makes both functions simpler. The controller can assume the data is already valid.

Outline

- 1 Middleware Pipeline Concept
- 2 Custom Logger Middleware
- 3 CORS Configuration
- 4 Input Validation Middleware
- 5 Global Error Handler & next(err)**
- 6 Correct Middleware Order
- 7 Recap

The Problem Without a Global Error Handler

Without error handler – crashes or hangs:

```
1 // Every route repeats the same try/catch
2 app.get('/api/courses/:id', async (req, res) => {
3   try {
4     const course = await Course.findById(
5       req.params.id
6     );
7     if (!course) {
8       return res.status(404)
9         .json({ error: 'Not found' });
10    }
11    res.json(course);
12  } catch (err) {
13    // Same error handling duplicated
14    // in EVERY route!
15    console.error(err);
16    res.status(500).json({
17      error: 'Server error'
18    });
19  }
20 });
21
22 // Same try/catch in every route...
23 app.post('/api/courses', async (req, res) => {
24   try {
25     // ...
26   } catch (err) {
27     // ...AGAIN
28     res.status(500).json({ error: 'Server error' });
29   }
30 })
```

With global error handler – clean:

```
1 // Routes become simple and clean
2 app.get('/api/courses/:id',
3   async (req, res, next) => {
4     try {
5       const course = await Course.findById(
6         req.params.id
7       );
8       if (!course) {
9         return res.status(404)
10           .json({ error: 'Not found' });
11       }
12       res.json(course);
13     } catch (err) {
14       next(err); // Send to global handler
15     }
16   }
17 );
18 // ONE global handler deals with ALL errors!
```



DRY principle

Don't Repeat Yourself. A global error handler lets you write error handling code once and use it for all routes.

The 4-Parameter Error Handler

W3Schools – Error-handling Middleware

Error-handling middleware is **special** because it takes **four parameters** instead of three: (err, req, res, next). They are defined **after** all other app.use() and route calls.

```
1 // middleware/errorHandler.js
2 const errorHandler = (err, req, res, next) => {
3   // 1. Log the full error (server-side only)
4   console.error('[Error]', err.stack);
5
6   // 2. Determine the status code
7   // (use custom statusCode or default 500)
8   const statusCode = err.statusCode || 500;
9
10  // 3. Build the response
11  const response = {
12    error: err.message || 'Internal Server Error'
13  };
14
15  // 4. In development: include the full stack
16  if (process.env.NODE_ENV === 'development') {
17    response.stack = err.stack;
18  }
19
20  res.status(statusCode).json(response);
21 };
22
23 module.exports = errorHandler;
```

```
1 // index.js -- MUST be LAST app.use()
2 const errorHandler =
3   require('./middleware/errorHandler');
4
5 app.use(errorHandler); // LAST!
```



Must be exactly 4 params

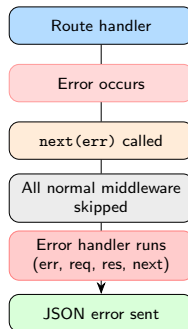
Express detects error-handling middleware by its **signature having exactly 4 parameters**. If you write (err, req, res) (missing next), Express will not treat it as an error handler.

500 response in development

```
{
  "error": "Cannot read properties",
```

next(err) – Passing Errors Down the Pipeline

```
1 // 1. Throw naturally (Express catches it)
2 app.get('/error-demo', (req, res) => {
3   throw new Error('Something went wrong!');
4   // Express catches this automatically
5 });
6
7 // 2. next(err) in try/catch (recommended)
8 app.get('/api/courses/:id',
9   async (req, res, next) => {
10     try {
11       const course = await findById(
12         req.params.id
13       );
14       res.json(course);
15     } catch (err) {
16       next(err); // pass to error handler
17     }
18   }
19 );
20
21 // 3. Custom error with status code
22 app.get('/api/courses/:id', (req, res, next) => {
23   const course = courses.find(
24     c => c.id === +req.params.id
25   );
26   if (!course) {
27     const err = new Error('Course not found');
28     err.statusCode = 404; // custom prop
29     return next(err); // pass to handler
30   }
31   res.json(course);
32 }
```



Outline

- 1 Middleware Pipeline Concept
- 2 Custom Logger Middleware
- 3 CORS Configuration
- 4 Input Validation Middleware
- 5 Global Error Handler & `next(err)`
- 6 Correct Middleware Order**
- 7 Recap

Why Middleware Order Matters

Express.js Routing Docs

One of the most important things about middleware in Express is the **order in which they are written**. Middleware is executed **top to bottom** as it is defined. If a middleware is placed after a route that matches, it will **never run** for that route.

Wrong order (common mistake):

```
1 const app = express();
2
3 // Routes defined BEFORE middleware!
4 app.get('/api/courses', handler); // <-- wrong
5
6 // These never apply to the route above
7 app.use(cors()); // too late
8 app.use(express.json()); // too late
9 app.use(logger); // too late
```

Problems caused by wrong order:

- req.body is undefined (JSON not parsed)
- CORS headers missing (React app blocked)
- Requests not logged
- 404 handler catches everything

Correct order:

```
1 require('dotenv').config();
2 const app = express();
3
4 // 1. CORS -- must be FIRST
5 app.use(cors(corsOptions));
6
7 // 2. Body parsing
8 app.use(express.json());
9
10 // 3. Custom middleware (logger, auth)
11 app.use(logger);
12
13 // 4. Routes
14 app.use('/api/courses', courseRoutes);
15 app.use('/api/users', userRoutes);
16
17 // 5. 404 handler (after all routes)
18 app.use((req, res) => {
19   res.status(404).json({
20     error: 'Route not found'
21   });
22 });
```


The Correct Middleware Stack – Final index.js

```
// index.js -- complete middleware-ordered server
require('dotenv').config();
const express = require('express');
const app = express();

// --- Import middleware ---
const cors = require('cors');
const logger = require('./middleware/logger');
const errorHandler = require('./middleware/errorHandler');

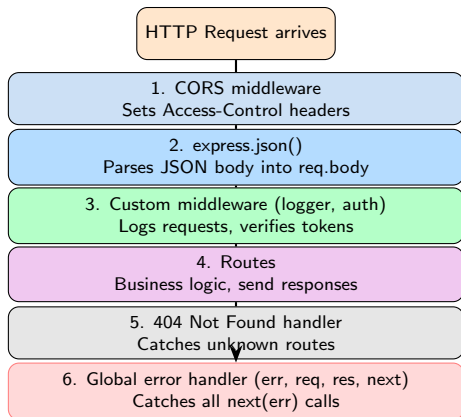
// --- Import routes ---
const courseRoutes = require('./routes/courseRoutes');
const userRoutes = require('./routes/userRoutes');

// =====
// STEP 1: CORS (first -- before anything else)
// =====
app.use(cors({ origin: process.env.FRONTEND_URL }));

// =====
// STEP 2: Body parsing middleware
// =====
app.use(express.json()); // parse JSON bodies
app.use(express.urlencoded({ extended: true })); // form data

// =====
// STEP 3: Custom middleware (logger, auth, etc.)
// =====
app.use(logger);
```

Complete Middleware Order Summary



Common Middleware Bugs and Fixes

Bug 1: req.body is undefined

```
1 // FIX: add express.json() BEFORE routes
2 app.use(express.json()); // <-- required
3 app.post('/api/courses', handler);
```

Bug 2: CORS error in browser

```
1 // FIX: add cors() as FIRST middleware
2 app.use(cors({
3   origin: process.env.FRONTEND_URL
4 }));
```

Bug 3: 404 handler catches everything

```
1 // FIX: 404 handler goes AFTER all routes
2 app.use('/api/courses', courseRoutes);
3 app.use('/api/users', userRoutes);
4 // 404 handler goes here -- LAST route
5 app.use((req, res) => {
6   res.status(404).json({ error: 'Not found' });
7 });
```

Bug 4: Error handler never fires

```
1 // WRONG -- missing 'next' param (3 args)
2 app.use((err, req, res) => {
3   res.status(500).json({ error: err.message });
4 });
5
6 // FIX -- must have exactly 4 params
7 app.use((err, req, res, next) => {
8   res.status(500).json({ error: err.message });
9 });
```

Bug 5: Error not passed forward

```
1 // WRONG -- error swallowed silently
2 app.get('/courses/:id', async (req, res) => {
3   try { /* ... */ }
4   catch (err) { console.error(err); }
5   // client hangs -- no response!
6 });
7
8 // FIX -- pass to error handler
9 } catch (err) { next(err); }
```



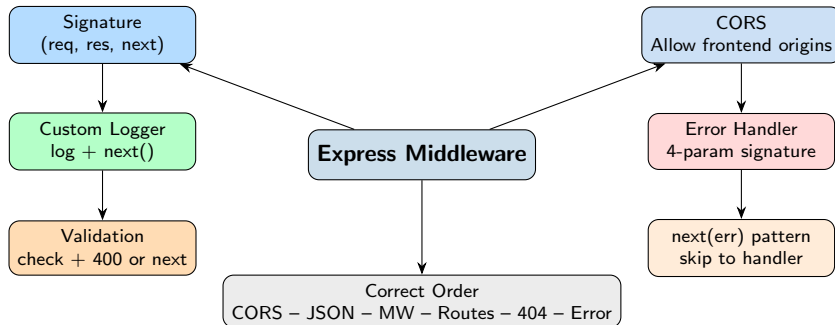
next() without calling

If middleware runs but neither calls next () nor sends a response, the request hangs indefinitely.

Outline

- 1 Middleware Pipeline Concept
- 2 Custom Logger Middleware
- 3 CORS Configuration
- 4 Input Validation Middleware
- 5 Global Error Handler & `next(err)`
- 6 Correct Middleware Order
- 7 Recap**

Middleware Pipeline – Concept Map



Lecture 12 – Key Takeaways

- 1 **Middleware** is a function with `(req, res, next)` that runs in a pipeline between request arrival and response
- 2 Calling `next()` passes control forward; calling `next(err)` jumps to the error handler; not calling either hangs the request
- 3 **Custom logger**: read `req.method` and `req.url`, log them, then call `next()` – or use Morgan
- 4 **CORS** is a browser security feature blocking cross-origin requests; configure it with `npm install cors + app.use(cors(options))`
- 5 Use `origin: '*'` only in development; specify exact frontend URLs in production
- 6 **Input validation middleware** checks `req.body` before the route handler runs; returns 400 on failure, calls `next()` on success
- 7 **Global error handler** must have exactly **4 parameters**: `(err, req, res, next)` and must be registered **last**
- 8 Use `next(err)` in route `try/catch` blocks instead of duplicating error code in every handler
- 9 **Correct order**: CORS → JSON parser → custom middleware → routes → 404 handler → error handler

Next Lecture: MongoDB & Mongoose – Connecting the API to a Real Database

Practice Exercises

- 1 Create a **logger middleware** in `middleware/logger.js` that prints the HTTP method, URL, and timestamp for every request. Register it with `app.use(logger)` before all routes.
- 2 Add **Morgan** to your project (`npm install morgan`). Compare its output to your custom logger. Use 'dev' format and observe the status code and response time it adds.
- 3 Run your Express API from Lecture 11 and try to call it from a React app running on `localhost:5173`. Observe the CORS error in the browser console. Then **fix it** with the `cors` package.
- 4 Configure CORS to allow only `http://localhost:5173` using an options object. Store the allowed origin in `.env` as `FRONTEND_URL` and read it with `process.env.FRONTEND_URL`.
- 5 Build a `validateCourse` middleware that checks for required fields (`title`, `dept`). Apply it to the POST and PUT routes. Test missing fields with Postman – confirm 400 is returned.